

Tuples

Tuples

- we've seen scalar types: `int` , `float` , `str` , `bool`
- we've also seen one compound type: `string`
- other compound data types include
 - indexed sequences of elements/objects
 - tuples (immutable)
 - lists (mutable)

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

- indexable ordered sequence of objects
 - can be any type, int, string, tuple, tuples of tuples, etc
- cannot change element values, **immutable**

```
1  te = () # empty tuple
2  ts = (1, 2, 3) # tuple of three integers
3  tt = (3,) # tuple of one integer, needs comma
4  tn = (1, "hello", (2, 3)) # tuple of mixed types
5
6  tn[0] # evaluates to 1
7  (2, 3) + (4, "five") # evaluates to (2, 3, 4, "five")
8  tn[1:2] # evaluates to ("hello",)
9  tn[1:3] # evaluates to ("hello", (2, 3))
10 len(tn) # evaluates to 3
11 tn[1] = 4 # TypeError, tuples are immutable
```

Tuples

A single-element tuple needs a trailing comma to distinguish it from a plain parenthesized expression

```
1 not_a_tuple = (3) # this is just the integer 3
2 is_a_tuple = (3,) # this is a tuple containing 3
```

Without the comma, Python just sees parentheses used for grouping, like in $(2 + 3) * 4$

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(len(seq)) # what is the length
2  print(seq[3])
3  print(seq[-1])
4  print(seq[3][0])
5  print(seq[4])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(len(seq)) # what is the length
2  print(seq[3])
3  print(seq[-1])
4  print(seq[3][0])
5  print(seq[4])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(len(seq)) # what is the length
2  print(seq[3])
3  print(seq[-1])
4  print(seq[3][0])
5  print(seq[4])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(len(seq)) # what is the length
2  print(seq[3])
3  print(seq[-1])
4  print(seq[3][0])
5  print(seq[4])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(len(seq)) # what is the length
2  print(seq[3])
3  print(seq[-1])
4  print(seq[3][0])
5  print(seq[4])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(seq[1])  
2  print(seq[-2:])  
3  print(seq[1:4:2])  
4  print(seq[:-1])  
5  print(seq[1:3])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(seq[1])  
2  print(seq[-2:])  
3  print(seq[1:4:2])  
4  print(seq[:-1])  
5  print(seq[1:3])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(seq[1])  
2  print(seq[-2:])  
3  print(seq[1:4:2])  
4  print(seq[:-1])  
5  print(seq[1:3])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(seq[1])  
2  print(seq[-2:])  
3  print(seq[1:4:2])  
4  print(seq[:-1])  
5  print(seq[1:3])
```

Indices and slicing

```
1  seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1  print(seq[1])  
2  print(seq[-2:])  
3  print(seq[1:4:2])  
4  print(seq[:-1])  
5  print(seq[1:3])
```

Indices and slicing

```
1 seq = (2, 'a', 4, (1,2)) # what is index 0
```

```
1 for item in seq:  
2     print(item)
```

Tuple uses

Unpacking, which is also a good way of swapping values

```
1  x = 10
2  y = 20
3  z = x
4  x = y
5  y = z
```

```
1  x = 10
2  y = 20
3  (x, y) = (y, x)
```

Because Python builds the entire tuple `(y, x)` on the right side *before* assigning it, there's no risk of overwriting a value before it's used, unlike the manual swap which needs a temporary variable `z`

Tuples uses

You can also use them to return more than one value from a function

```
1 def quotient_and_remainder(x, y):
2     q = x // y
3     r = x % y
4     return (q, r) # one object, multiple values
5
6 both = quotient_and_remainder(10, 3)
7 (quot, rem) = quotient_and_remainder(10, 3)
```

- `both` is the whole tuple `(3, 1)`
- `quot` and `rem` are the unpacked individual values, `3` and `1`